COL1101

LECTURE-4

JULY 30, 2026

Last Class: Analyze runtime complexity of algorithm as a function of input size

Key ideas:

- Count # of primitive steps
- Worst-case input * (e.g. quicksort)
- Ignore constant factors and lower order terms

machine dependent

irrelevant for larger n

(Asymptotic analysis: large values of n)

Last Class: Big-Oh Notation

Definition: Let $f, g : \mathbb{N} \to \mathbb{R}^+$ be two functions. $f(n)$ is $O(g(n))$ if $\exists$ constants c and n_0 : $f(n) \leq c\, g(n)$ $\forall\, n \geq n_0$

$f(n) = n^2 + 5n + 3$ $\rightarrow$ $g(n)$ could be n^2, n^3, ...

$f(n)$ is $O(n^2)$

$f(n)$ is $n + \log n$ $\qquad$ $f(n)$ is $O(n)$

$$\lim_{n\to\infty} \frac{n+\log n}{n} \rightarrow \lim_{n\to\infty} \frac{1 + \frac{\log n}{n}}{1} = 1$$

Exercise: $\lim_{n\to\infty} \frac{f(n)}{g(n)}$ is finite then $f(n)$ is $O(g(n))$

Working Rule: f is $O(g)$ if $\lim_{n\to\infty} \frac{f(n)}{g(n)}$ is finite

NOT iff

MYTHS ABOUT BIG-OH → gives upper bound

1. Big-O is the running time
2. Big-O ignores constants, so constants don't matter
3. n is not $O(n^2)$

n is $O(n^2)$

OTHER NOTATIONS

1. **Big - Omega** : Let $f, g : \mathbb{N} \to \mathbb{R}^+$ be functions

$f(n)$ is $\Omega(g(n))$, if $\exists$ constants $c > 0$ & n_0 $\therefore f(n) \geq c\, g(n)$ $\forall\, n \geq n_0$

E.g. n^2 is $\Omega(n)$

Exercise : $f(n)$ is $\Omega(g(n))$ iff $g(n)$ is $O(f(n))$

OTHER NOTATIONS

1. **Big - Omega** : let $f, g : \mathbb{N} \to \mathbb{R}^+$ be functions

$f(n)$ is $\Omega(g(n))$, if $\exists$ constants $c > 0$ & n_0 $\therefore f(n) \geq c\, g(n)$ $\forall\ n \geq n_0$

2. **Big - Theta** : $f(n)$ is $\Theta(g(n))$ is $\exists$ constants c_1, c_2, n_0 :

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \forall\ n \geq n_0$$

$f(n)$ is $\Theta(g(n))$ iff $f(n)$ is $O(g(n))$ & $f(n)$ is $\Omega(g(n))$

Working rule : f is $\Theta(g)$ if $\lim_{n \to \infty} \frac{f(n)}{g(n)}$ is finite and non-zero

SOME MORE NOTATIONS

$f(n)$ is $o(n)$ small-oh

$f(n)$ is $\omega(n)$ small-omega

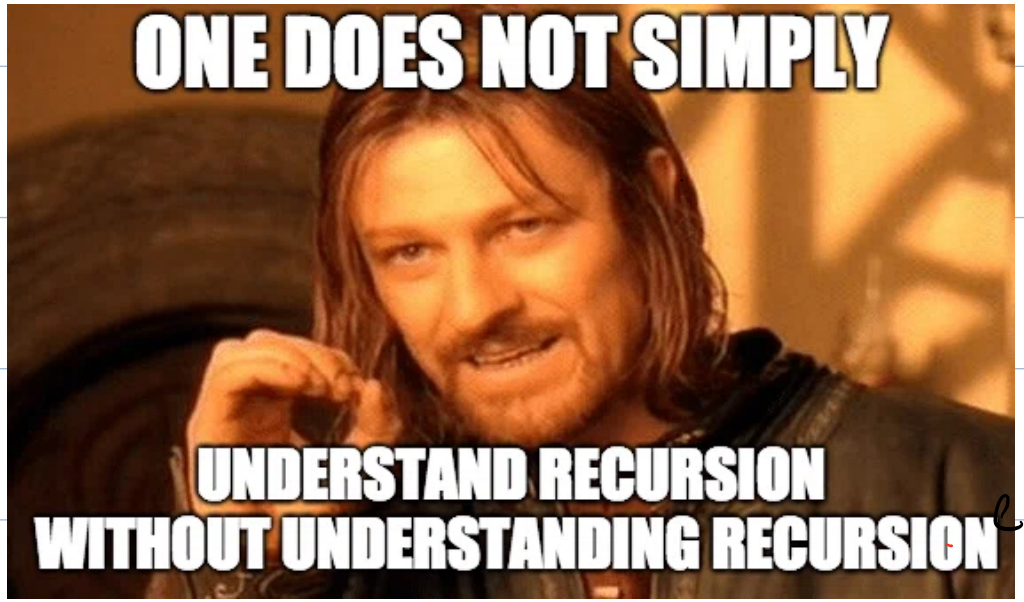
Read about it

Chapter 3 in the

Today's Class

How to quantify the runtime complexity of recursive algorithms?

Recursion Revisited



Solve a problem by calling the same function on smaller versions of the problem ; until the version is small enough to answer directly

Example: Calculate Factorial Using Recursion

```
def factorial(n):
    # base case
    if n == 0:
        return 1
    # recursive case
    return n * factorial(n - 1)
```

c c

How to find runtime of this function?

Question: How fast is this algorithm?

Step 1: count # of primitive steps

Runtime Complexity Of Factorial Function

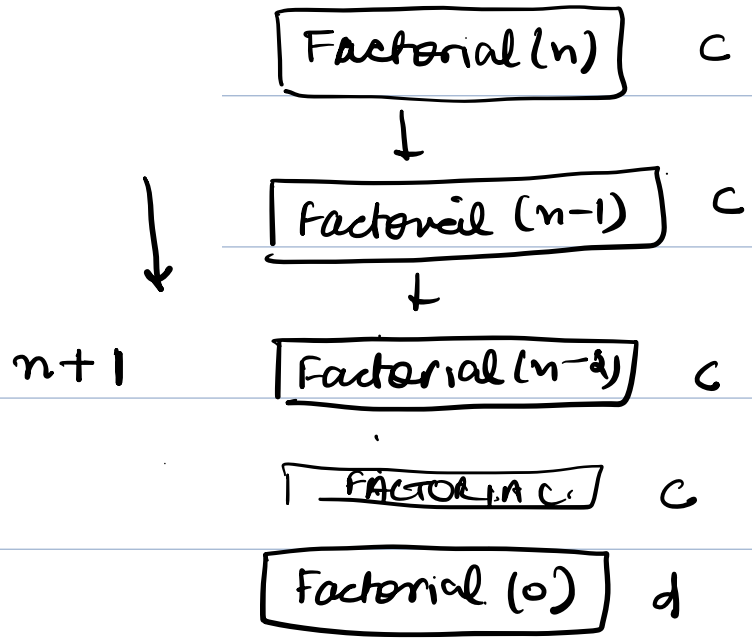
```
def factorial(n):
    # base case
    if n == 0:
        return 1
    # recursive case
    return n * factorial(n - 1)
```

c

c

$T(n) = T(n-1) + O(1)$ → Recurrence Relation

Recurrence Trace:



Runtime:

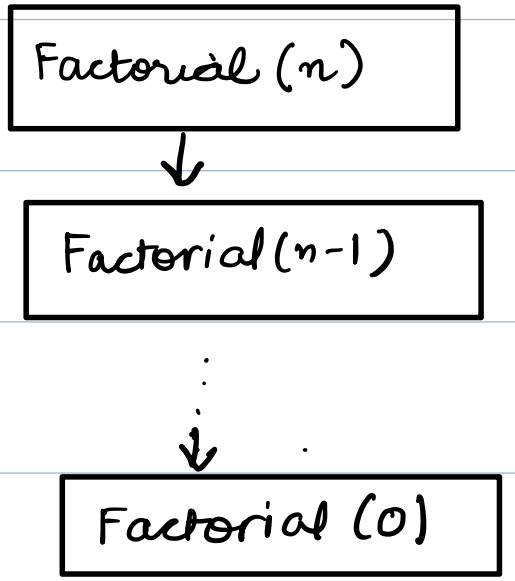
$nc + d$

$O(n)$ $\Theta(n)$

Runtime Complexity Of Factorial Function

```
def factorial(n):
    # base case
     if n == 0:
        return 1
    # recursive case
     return n * factorial(n - 1)
```

Method 1: Recursion trace



Runtime Complexity Of Factorial Function

```
def factorial(n):
    # base case
    if n == 0:
        return 1
    # recursive case
    return n * factorial(n - 1)
```

Method 2: Solve the recurrence relation algebraically

$T(n) = T(n-1) + c$ & $T(0) = d$

$T(n) = \overbrace{T(n-2) + c}^{\downarrow} + c$

$= T(n-2) + 2c$

$= T(n-3) + 3c$

$= T(n-k) + kc$

$T(n) = T(0) + nc \rightarrow \Theta(n)$

$= nc + d$

Is Time The Only Thing An Algorithm Consume?

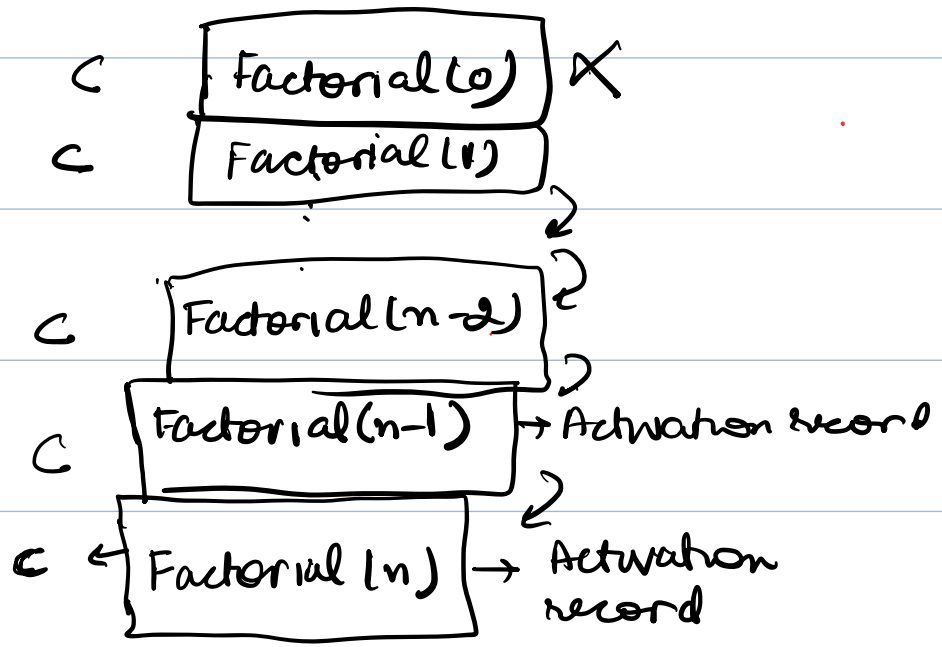
```
>>> factorial(1000)
Traceback (most recent call last):
  File "<python-input-3>", line 1, in <module>
    factorial(1000)
    ~~~~~~~~~^^^^^^
  File "<python-input-0>", line 4, in factorial
    return n*factorial(n-1)
             ~~~~~~~~~^^^^^
  File "<python-input-0>", line 4, in factorial
    return n*factorial(n-1)
             ~~~~~~~~~^^^^^
  File "<python-input-0>", line 4, in factorial
    return n*factorial(n-1)
             ~~~~~~~~~^^^^^
  [Previous line repeated 988 more times]
RecursionError: maximum recursion depth exceeded
```

Each function call is associated with an

activation record or stack frame.

WHAT IS AN ACTIVATION RECORD?

A block of memory for a single function call, holding the call parameters, local variables and a "return address"



Space used by the stack frame grows like nc

$\Theta(n$

DEFINITION : SPACE COMPLEXITY

The amount of extra memory algorithm uses a function of the input size - beyond the input size

Described with the same Big-O / Ω / Θ tools we use for time

EXAMPLE -2

RUNTIME COMPLEXITY OF RECURSIVE BINARY SEARCH

```
def bin_search(a, lo, hi, key):
    if lo > hi:              # not found
        return -1
    mid = (lo + hi) // 2
    if a[mid] == key:
        return mid
    if a[mid] < key:
        # search right
        return bin_search(a, mid + 1, hi, key)
    # search left
    return bin_search(a, lo, mid - 1, key)
```

Step 1

Recurrence relation $n = 2^k$

$$T(n) = T\left(\frac{n}{2}\right) + c$$

Recurrence trace

$[n] \cdot c$

$\downarrow$

$\left[\frac{n}{2}\right] c$

$\downarrow$

$\left[\frac{n}{4}\right] c$

$\vdots$

$[1]\ c$

$\log_2 n$ steps

$c \log_2 n$

$\Theta(\log_2 n)$

Runtime Complexity Of Recursive Binary Search

```
def bin_search(a, lo, hi, key):
    if lo > hi:              # not found
        return -1
    mid = (lo + hi) // 2
    if a[mid] == key:
        return mid
    if a[mid] < key:
        # search right
        return bin_search(a, mid + 1, hi, key)
    # search left
    return bin_search(a, lo, mid - 1, key)
```

Recurrence tree

$T(0) = 1$

$T(1) = 1$

$$T(n) = T\left(\frac{n}{2}\right) + c$$

$$T(n) = T\left(\frac{n}{2^2}\right) + \underbrace{c + c}_{2c}$$

$$T\left(\frac{n}{2^3}\right) + 3c$$

$$= T\left(\frac{n}{2^i}\right) + ic$$

$i = \log_2 n$

$= T(1) + \log_2 n \times c$

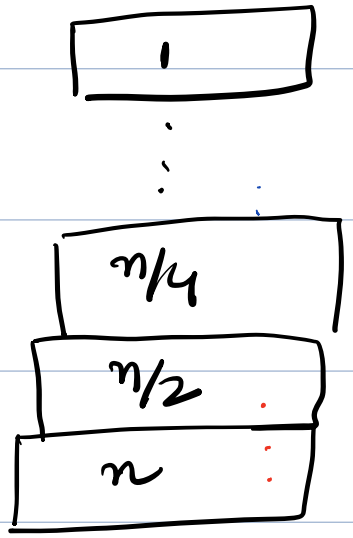
$\Theta(\log_2 n)$

What about space complexity?

Space Complexity Of Recursive Binary Search

```
def bin_search(a, lo, hi, key):
    if lo > hi:                 # not found
        return -1
    mid = (lo + hi) // 2
    if a[mid] == key:
        return mid
    if a[mid] < key:
        # search right
        return bin_search(a, mid + 1, hi, key)
    # search left
    return bin_search(a, lo, mid - 1, key)
```

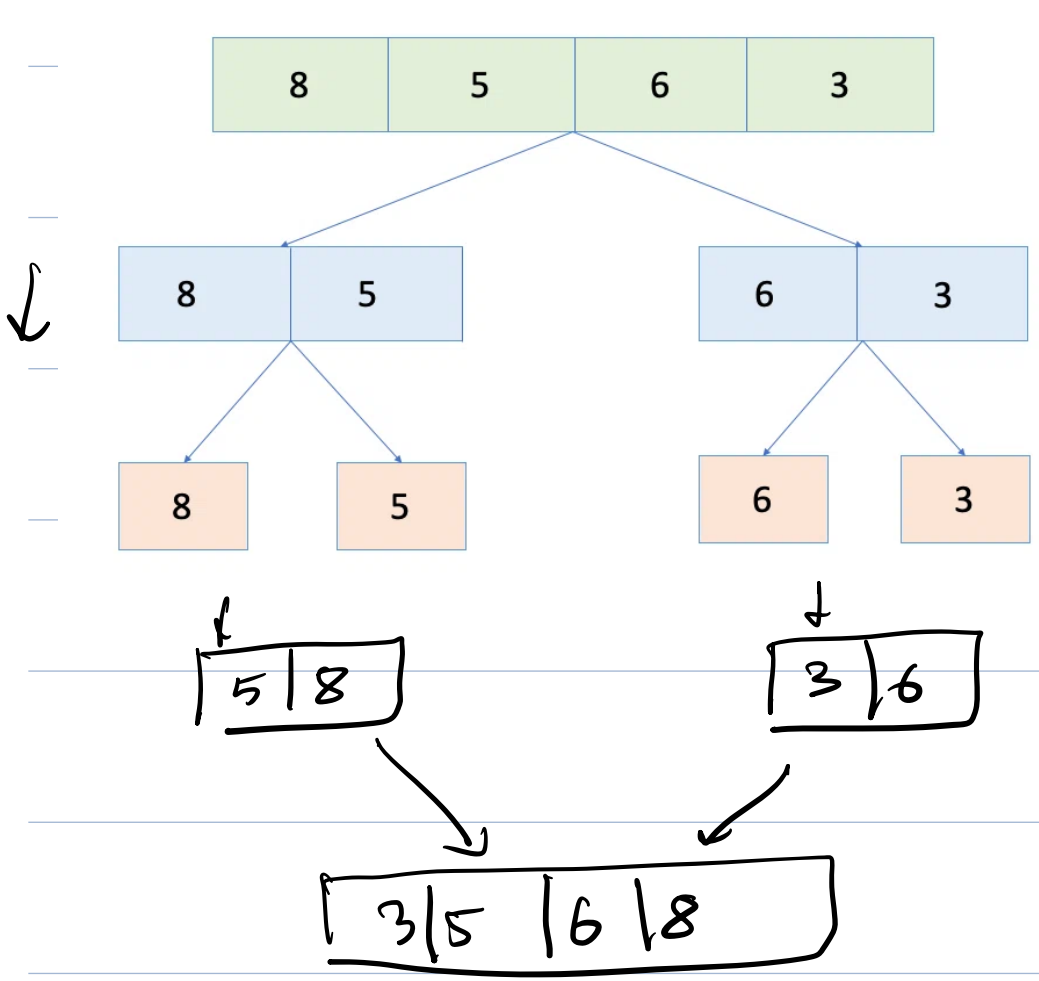
Recurrence tree



Space complexity: $\Theta(\log_2 n)$

Tail Recursion

Example #3: Merge Sort



Divide and conquer strategy

Example #3: Merge Sort

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left  = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)
```

Merge Sort : Time Complexity

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left  = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)
```

Method 2 : Solving the recurrence algebriacally

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

$$T(1) = d$$

Merging Two Sorted Lists

$n/2$ $n/2$

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]);
            i += 1
        else:
            result.append(right[j]);
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Merge Sort : Time Complexity

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left  = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)
```

Step 1: Recurrence relation

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

